Utilize Public Key Cryptography

RSA Key Pairs · Asymmetric Encryption · A Simple PKI · Secure Key Exchange

From-scratch Python implementations (RSA, OAEP, X25519, HKDF) verified against RFC test vectors, the production cryptography library and the OpenSSL command line — 19 of 19 tests passed

Project Name	Healthcare IAM Operations Simulation
Organization	Horizon Valley Medical Center (HVMC)
Rubric	Cryptography 1 — Utilize Public Key Cryptography
Code	`cryptography_1/public_key_crypto/` : `rsa.py`, `key_management.py`, `pki.py`, `key_exchange.py`, `test_and_demo.py`, `pki/`, `results.txt`, `openssl_checks.txt`
Environment	Python 3.9.6 on Darwin arm64, 2026-10-26 13:37; cryptography package: 47.0.0; LibreSSL 3.3.6
Prepared By	Sabitha S Karanavar
Report Date	October 26, 2026

Rubric Cross-Reference

Rubric requirement	Section	Evidence
Generation and management of RSA key pairs	2, 3	RSA-2048/3072/4096 generated from scratch (Miller-Rabin, FIPS 186-5 checks) and accepted by the library; encrypted PKCS #8 storage (mode 0600), key inventory with cryptoperiods, rotation, retirement and destruction
Implementation of basic asymmetric encryption	4	Textbook RSA weaknesses shown; RSA-OAEP (RFC 8017) from scratch; interop with the library and OpenSSL in both directions; hybrid RSA-OAEP + AES-256-GCM for patient records
Setup of a simple PKI environment	5	Two-tier PKI: offline root CA, issuing CA, CSR policy checks, TLS server and clinician client certificates, register, CRLs; path validation by 3 independent verifiers
Demonstration of secure key exchange	6	X25519 (RFC 7748) and HKDF (RFC 5869) from scratch; MITM on unauthenticated exchange; certificate-authenticated handshake with AES-256-GCM channel; forward secrecy vs RSA key transport

1. Introduction

Public key (asymmetric) cryptography uses a **key pair**: a **public key** anyone may have and a **private key** only its owner holds. What the public key encrypts, only the private key can decrypt; what the private key signs, anyone can verify with the public key. This solves the problem symmetric cryptography cannot solve alone — **how two parties who have never met agree on a secret key** — and it lets HVMC prove *who* is on the other end of a connection.

At HVMC this protects the patient portal (`portal.hvmc.example`), clinician access to the EHR, and links between systems such as the EHR and the laboratory system. HIPAA requires **person or entity authentication** (§164.312(d)) and **transmission security** (§164.312(e)(1)); certificates and authenticated key exchange are the standard way to meet both.

Following the earlier HVMC cryptography reports, the core algorithms are written **from scratch** in Python so the mathematics is visible, then checked three ways: against **official RFC test vectors**, against the production **cryptography** library (47.0.0), and against the **OpenSSL command line** (LibreSSL 3.3.6). Certificates, PEM storage and AES-GCM use the library, as production code should.

19/19	6	4	19	3
unit tests passed	RSA key pairs in the inventory	certificates in a 2-tier PKI	path-validation checks passed	independent verifiers agree

File	What it contains
`rsa.py`	Miller-Rabin, prime and key generation, CRT decryption with blinding, I2OSP/OS2IP, MGF1, RSA-OAEP, PKCS #1 v1.5 signature verification
`key_management.py`	Key policy (NIST SP 800-57), encrypted PKCS #8 storage, fingerprints, key inventory and lifecycle, hybrid encryption, rotation
`pki.py`	Root CA, issuing CA, CSRs, issuance policy, certificate register, CRLs, RFC 5280-style path validation
`key_exchange.py`	X25519, HKDF, unauthenticated vs certificate-authenticated handshake, AES-256-GCM record layer, RSA key transport
`test_and_demo.py`	19 unit tests and 5 demonstrations; writes `results.txt`, `results.json`, `openssl_checks.txt` and `pki/`

2. RSA Key Pair Generation

2.1 How RSA works

Step	Mathematics	Why
1. Pick two primes	Random primes p, q, each half the key size	The security rests on factoring n back into p and q being infeasible
2. Modulus	$n = p \cdot q$ (2048, 3072 or 4096 bits)	Public; its size is the key size
3. Carmichael function	$\lambda(n) = \text{lcm}(p-1, q-1)$	The smallest exponent with $a^{\lambda(n)} \equiv 1 \pmod n$
4. Public exponent	$e = 65537 = 2^{16}+1$, with $\gcd(e, \lambda(n)) = 1$	Fast to use (17 squarings), and large enough to avoid small-exponent attacks
5. Private exponent	$d = e^{-1} \bmod \lambda(n)$ (extended Euclid)	Then $(m^e)^d \equiv m \pmod n$ for every m
6. CRT values	$dP = d \bmod (p-1)$, $dQ = d \bmod (q-1)$, $qInv = q^{-1} \bmod p$	Decryption with the Chinese Remainder Theorem is about 3–4× faster

Worked example (the classic textbook key, reproduced by `test_textbook_example`): $p = 61$, $q = 53$, $n = 3233$, $\lambda(n) = \text{lcm}(60, 52) = 780$, $e = 17$, $d = 17^{-1} \bmod 780 = 413$. Encrypting $m = 65$ gives $65^{17} \bmod 3233 =$ **2790**, and $2790^{413} \bmod 3233 =$ **65**. (Textbooks often show $d = 2753$, computed with $\phi(n) = 3120$; both work, and λ gives the smaller exponent that RFC 8017 allows.)

2.2 Generating real keys

Prime generation. Candidates come from Python's `secrets` CSPRNG with the top two bits set (so $p \cdot q$ has exactly the requested size) and the low bit set (odd). Trial division by the primes below 2000 discards most candidates cheaply; the survivors face **40 rounds of Miller-Rabin** (error probability $\le 2^{-80}$). The test confirms Miller-Rabin rejects Carmichael numbers (561, 1105, 1729, 2465, 41041), which fool the simpler Fermat test, and accepts the Mersenne prime $2^{521}-1$.

Key checks (FIPS 186-5). The generator rejects keys below **2048 bits** (HVMC policy), primes that are too close ($|p - q|$ must exceed $2^{\text{nlen}/2 - 100}$, otherwise Fermat factoring finds them), and a private exponent d smaller than $2^{\text{nlen}/2}$ (Wiener's attack).

```
def generate_prime(bits, e=65537):
    """Random prime of exactly `bits` bits with the top two bits set (so p*q has
    exactly 2*bits bits) and gcd(p - 1, e) = 1 (so e is invertible)."""
    while True:
        cand = secrets.randbits(bits) | (0b11 << (bits - 2)) | 1
        if any(cand % p == 0 for p in SMALL_PRIMES):        # cheap trial division first
            continue
        if math.gcd(cand - 1, e) == 1 and is_probable_prime(cand):
            return cand
```

```
def generate_keypair(bits=2048, e=65537):
    """RSA key generation. Rejects p, q that are too close together (FIPS 186-5
    requires |p - q| > 2^(nlen/2 - 100), otherwise Fermat factoring finds them)."""
    if bits < 2048:
        raise ValueError("HVMC policy: RSA keys must be at least 2048 bits")
    half = bits // 2
    while True:
        p, q = generate_prime(half, e), generate_prime(bits - half, e)
        if abs(p - q) <= 1 << (half - 100):
            continue
        key = RSAPrivateKey(max(p, q), min(p, q), e)
        if key.n.bit_length() == bits and key.d > 1 << half:   # small d is insecure (Wiener)
            return key
```

Key size	Time to generate (from scratch)	Security strength (NIST SP 800-57)	HVMC use
RSA-2048	0.36 s	112 bits — acceptable through 2030	TLS server and clinician client keys
RSA-3072	1.08 s	128 bits — acceptable beyond 2030	Issuing CA; EHR data-encryption key
RSA-4096	3.16 s	~150 bits	Offline root CA (10-year lifetime)

Proof that the keys are real. Every key in this project — including the CA keys — comes from this generator. `to_library()` loads p, q, d and the CRT values into the **cryptography** package, which rejects inconsistent RSA numbers; the library then signs and verifies with the key, and OpenSSL decrypts with it (section 4.4).

2.3 Decryption with CRT and blinding

`decrypt_raw` uses the CRT values and **base blinding**: the ciphertext is first multiplied by r^e for a fresh random r, and the result by r^{-1}. The private-key operation therefore never runs on a value the attacker chose, which defeats timing attacks that measure decryption time. The test checks that the CRT result, with and without blinding, equals the plain $c^d \bmod n$.

```
    def decrypt_raw(self, c, blind=True):
        """RSADP with the Chinese Remainder Theorem and base blinding.
        Blinding multiplies c by r^e first, so the time taken does not depend on
        the ciphertext an attacker chose (a timing side-channel defence)."""
        if not 0 <= c < self.n:
            raise ValueError("ciphertext representative out of range")
        if blind:
            while True:
                r = secrets.randbelow(self.n - 2) + 2
                if math.gcd(r, self.n) == 1:
                    break
            c = c * pow(r, self.e, self.n) % self.n
        m1, m2 = pow(c, self.dp, self.p), pow(c, self.dq, self.q)
        h = self.qinv * (m1 - m2) % self.p
        m = m2 + h * self.q
        return m * modinv(r, self.n) % self.n if blind else m
```

3. RSA Key Management

A key is only as safe as the way it is stored, used and retired. `key_management.py` enforces an HVMC key policy based on **NIST SP 800-57 Part 1 Rev. 5**.

3.1 Key policy

Key purpose	Algorithm	Cryptoperiod	Storage (lab)	Production equivalent
Root CA	RSA-4096	10 years	Encrypted file, offline	Offline HSM; used only to sign the issuing CA and its CRL
Issuing CA	RSA-3072	5 years	Encrypted file, CA host	Azure Key Vault Managed HSM (FIPS 140-3 Level 3)
TLS server	RSA-2048	397 days	Encrypted file, web server	Key Vault certificate with auto-renewal
Client authentication	RSA-2048	365 days	Encrypted file, user device	TPM / Windows Hello or a smart card (non-exportable)
Data encryption (EHR)	RSA-3072	2 years	Encrypted file, key service	Key Vault key; wrap/unwrap only, no export

3.2 Protected storage

Private keys are written as **encrypted PKCS #8** (`BEGIN ENCRYPTED PRIVATE KEY`). OpenSSL's `asn1parse` shows the scheme: **PBES2** with **PBKDF2-HMAC-SHA256** (random 16-byte salt) and **AES-256-CBC**. Files are created with mode **0600** inside a **0700** directory, so other accounts on the machine cannot read even the encrypted form. The passphrase comes from the `HVMC_PKI_PASSPHRASE` environment variable or a random 256-bit value in `pki/private/.passphrase`; the whole `pki/private/` folder is **git-ignored**, so no private key or passphrase is published with this report.

> **Limitation.** The library uses only 2,048 PBKDF2 iterations for PEM encryption; that is fine for a random 256-bit passphrase but weak for a human-chosen one. In production HVMC keeps private keys in an HSM or Key Vault, where they cannot be exported at all.

Fingerprints. Each key is identified by the SHA-256 hash of its DER *SubjectPublicKeyInfo* — the same value used for certificate and SSH key pinning. A fingerprint identifies a key without revealing anything secret, so it can appear in tickets, inventories and change records.

3.3 Key inventory and lifecycle

Every key pair is recorded in `pki/key_inventory.json` with its owner, purpose, dates, status and history. Status only moves forward: **active** → **retired** (decrypt/verify only) → **destroyed**, or **active** → **compromised**. `KeyStore.require()` is called before every use and refuses to encrypt with a retired or expired key, or to use a destroyed one.

Key ID	Alg.	Purpose	Owner	Created	Expires	Status	Fingerprint
`hvmc-root-ca-r1`	RSA-4096	root-ca	Information Security (offline)	2026-10-26	2036-10-23	**active**	`1893a7eca7cdffe4…`
`hvmc-issuing-ca-1`	RSA-3072	issuing-ca	Information Security	2026-10-26	2031-10-25	**active**	`8a838eb2dc8b84c4…`
`portal-tls-2026`	RSA-2048	tls-server	IT Infrastructure (Kevin O'Brien)	2026-10-26	2027-11-27	**active**	`b843035b68f540df…`
`sarah-chen-client-2026`	RSA-2048	client-auth	Sarah Chen (Doctors)	2026-10-26	2027-10-26	**active**	`0ffab9ec1a3f1bcd…`
`ehr-data-key-v1`	RSA-3072	data-encryption	EHR key service	2026-10-26	2028-10-25	**destroyed**	`749e29f1c29e3f5b…`
`ehr-data-key-v2`	RSA-3072	data-encryption	EHR key service	2026-10-26	2028-10-25	**active**	`018f6dc852fac4a0…`

```
    def require(self, key_id, use, now=None):
        """Enforce the key policy before every use."""
        rec = self.keys[key_id]
        now = now or dt.datetime.now(UTC)
        if use == "encrypt":
            if rec["status"] != "active":
                raise KeyPolicyError(f"{key_id} is {rec['status']}: new encryption not allowed")
            if now > dt.datetime.fromisoformat(rec["expires"]):
                raise KeyPolicyError(f"{key_id} expired on {rec['expires'][:10]}")
        elif use == "decrypt" and rec["status"] not in ("active", "retired"):
            raise KeyPolicyError(f"{key_id} is {rec['status']}: no longer usable")
        return rec
```

3.4 Rotation without re-encrypting the data

RSA is slow and can only encrypt a short message, so HVMC uses **envelope (hybrid) encryption** (section 4.3): the record is encrypted with a random AES-256-GCM **data key**, and only that data key is wrapped with RSA-OAEP. To rotate the RSA key, `rewrap()` unwraps the data key with the old key and wraps it with the new one — the bulk ciphertext is unchanged. The demo rotates `ehr-data-key-v1` to `v2` under change **HVMC-PKI-CHG-402**, shows that v1 can no longer encrypt but can still decrypt old envelopes while it is retired, then destroys v1 once every envelope is rewrapped.

```
=== Demo 2: Key management - storage, inventory, rotation ===
  key id                  algorithm  purpose         expires     status   fingerprint (SHA-256 of SPKI)
  hvmc-root-ca-r1         RSA-4096   root-ca         2036-10-23  active   1893a7eca7cdffe4e84352ed…
  hvmc-issuing-ca-1       RSA-3072   issuing-ca      2031-10-25  active   8a838eb2dc8b84c4a22bcb6e…
  portal-tls-2026         RSA-2048   tls-server      2027-11-27  active   b843035b68f540dfa2207835…
  sarah-chen-client-2026  RSA-2048   client-auth     2027-10-26  active   0ffab9ec1a3f1bcd62991b53…
  -rw-------  private/hvmc-issuing-ca-1.key.pem         -----BEGIN ENCRYPTED PRIVATE KEY-----
  -rw-------  private/hvmc-root-ca-r1.key.pem           -----BEGIN ENCRYPTED PRIVATE KEY-----
  -rw-------  private/portal-tls-2026.key.pem           -----BEGIN ENCRYPTED PRIVATE KEY-----
  -rw-------  private/sarah-chen-client-2026.key.pem    -----BEGIN ENCRYPTED PRIVATE KEY-----
  load with a wrong passphrase -> refused (ValueError: Incorrect password, could not decrypt key)
  hybrid encryption of a 169-byte discharge record: RSA-OAEP-wrapped key 384 bytes + AES-256-GCM ciphertext 185 bytes
  rotation: data key rewrapped under v2; AES ciphertext unchanged = True
  encrypt with retired v1 -> refused: ehr-data-key-v1 is retired: new encryption not allowed
  decrypt old envelope with retired v1 still works: True
  after destruction, v1 envelope -> ehr-data-key-v1 is destroyed: no longer usable; v2 envelope decrypts: True
```

4. Basic Asymmetric Encryption

4.1 Why "textbook" RSA is not enough

Raw RSA ($c = m^e \bmod n$) works mathematically but is insecure on its own. The demo shows two problems on HVMC's real 2048-bit portal key:

Weakness	What the demo shows	Consequence
Deterministic	Encrypting "APPROVE 5" twice gives the same ciphertext	An eavesdropper can tell when the same message is sent again, or guess-and-check short messages such as "yes"/"no"
Malleable	Multiplying the ciphertext by 2^e makes it decrypt to $2 \cdot m$	An attacker can change a value (for example a dose or an amount) without the key and without detection
Size-limited	A message must be smaller than n	Large records cannot be encrypted directly

4.2 RSA-OAEP from scratch (RFC 8017 §7.1)

Optimal Asymmetric Encryption Padding fixes the first two problems. Before RSA is applied, the message is mixed with a fresh random **seed** through two rounds of the mask-generation function **MGF1** (built on SHA-256):

```
DB = lHash(label) ‖ 00…00 ‖ 01 ‖ M                  (k − 33 bytes)
maskedDB   = DB   ⊕ MGF1(seed, k − 33)
maskedSeed = seed ⊕ MGF1(maskedDB, 32)
EM = 00 ‖ maskedSeed ‖ maskedDB     →     c = EM^e mod n
```

Because the seed is new every time, the same message encrypts differently each time; because any change to the ciphertext scrambles the whole decoded block, a modified ciphertext fails the checks (lHash, the 0x01 separator and the leading zero byte). Decryption checks everything first and then raises **one generic error**, so the error does not reveal which check failed (Manger's attack exploits that difference). With RSA-2048 and SHA-256, OAEP carries at most 256 − 2·32 − 2 = **190 bytes**.

```
def oaep_encrypt(pub, message, label=b"", hash_name="sha256", _seed=None):
    """RSAES-OAEP-ENCRYPT (RFC 8017 section 7.1.1).
    EM = 0x00 || maskedSeed || maskedDB, where DB = lHash || PS || 0x01 || M."""
    h_len = hashlib.new(hash_name).digest_size
    k = pub.k
    if len(message) > k - 2 * h_len - 2:
        raise ValueError(f"message too long: max {k - 2 * h_len - 2} bytes for this key")
    l_hash = hashlib.new(hash_name, label).digest()
    ps = b"\x00" * (k - len(message) - 2 * h_len - 2)
    db = l_hash + ps + b"\x01" + message
    seed = _seed if _seed is not None else secrets.token_bytes(h_len)   # fresh randomness every time
    masked_db = _xor(db, mgf1(seed, k - h_len - 1, hash_name))
    masked_seed = _xor(seed, mgf1(masked_db, h_len, hash_name))
    em = b"\x00" + masked_seed + masked_db
    return i2osp(pub.encrypt_raw(os2ip(em)), k)

def oaep_decrypt(priv, ciphertext, label=b"", hash_name="sha256"):
    """RSAES-OAEP-DECRYPT (RFC 8017 section 7.1.2)."""
    h_len = hashlib.new(hash_name).digest_size
    k = priv.k
    if len(ciphertext) != k or k < 2 * h_len + 2:
```

```
        raise DecryptionError("decryption error")
    c = os2ip(ciphertext)
    if c >= priv.n:
        raise DecryptionError("decryption error")
    em = i2osp(priv.decrypt_raw(c), k)
    y, masked_seed, masked_db = em[0], em[1:1 + h_len], em[1 + h_len:]
    seed = _xor(masked_seed, mgf1(masked_db, h_len, hash_name))
    db = _xor(masked_db, mgf1(seed, k - h_len - 1, hash_name))
    l_hash = hashlib.new(hash_name, label).digest()
    rest = db[h_len:]
    sep = rest.find(b"\x01")
    zeros_ok = sep >= 0 and rest[:sep] == b"\x00" * sep
    # check everything, then fail once
    if not (secrets.compare_digest(db[:h_len], l_hash) and y == 0 and zeros_ok):
        raise DecryptionError("decryption error")
    return rest[sep + 1:]
```

4.3 Hybrid encryption for patient records

A discharge record (169 bytes of JSON) is encrypted with **AES-256-GCM** under a random data key, with the MRN as *associated data* so the ciphertext cannot be moved to another patient. Only the 32-byte data key is encrypted with RSA-OAEP (the from-scratch implementation), with the key ID as the OAEP **label**, which binds the wrapped key to the RSA key it was made for. The envelope (`sample_envelope_v2.json`) records the key ID, so the right private key — and only that key — can open it.

```
def hybrid_encrypt(store, key_id, plaintext, aad=b""):
    """RSA is slow and can only encrypt k - 66 bytes (190 bytes for RSA-2048 with OAEP-SHA-256),
    so HVMC encrypts the record with a random AES-256-GCM data key and wraps only that 32-byte
    key with RSA-OAEP (the from-scratch implementation)."""
    store.require(key_id, "encrypt")
    pub = rsa.from_library_public(store.public(key_id))
    dek = AESGCM.generate_key(256)
    nonce = secrets.token_bytes(12)
    return {
        "key_id": key_id, "alg": "RSA-OAEP-256 + A256GCM",
        "wrapped_key": _b64(rsa.oaep_encrypt(pub, dek, label=key_id.encode())),
        "nonce": _b64(nonce), "aad": _b64(aad),
        "ciphertext": _b64(AESGCM(dek).encrypt(nonce, plaintext, aad)),
    }
```

4.4 Results and interoperability

```
=== Demo 3: Asymmetric encryption - textbook RSA vs RSA-OAEP ===
  textbook RSA is deterministic: same message twice -> identical ciphertext = True
  textbook RSA is malleable: multiply ciphertext by 2^e -> decrypts to 2*m = True (attacker changed the value without the key)
  OAEP is randomized: two encryptions differ = True; both decrypt correctly = True
  OAEP rejects a modified ciphertext with a single generic 'decryption error'
  size limit: RSA-2048 + OAEP-SHA-256 carries at most 190 bytes -> HVMC uses hybrid encryption
  interop: OpenSSL decrypts from-scratch OAEP = True; from-scratch decrypts OpenSSL OAEP = True
```

Interop check	Result
From-scratch OAEP ciphertext → decrypted by the **cryptography** library	**PASS**
Library OAEP ciphertext → decrypted by the from-scratch code	**PASS**
From-scratch OAEP ciphertext → decrypted by **OpenSSL** `pkeyutl`	**PASS**
OpenSSL OAEP ciphertext → decrypted by the from-scratch code	**PASS**

LibreSSL 3.3's `pkeyutl` ignores `-passin`, so the encrypted key is first opened with `openssl pkey` and piped in; the unencrypted key exists only in the pipe and never on disk:

```
$ openssl pkey -in private/portal-tls-2026.key.pem -passin file:private/.passphrase | openssl pkeyutl -decrypt -inkey /dev/stdin -pkeyopt
rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256 -in from_scratch.bin
HVMC-004417 lab result key

$ openssl pkeyutl -encrypt -pubin -inkey public/portal-tls-2026.pub.pem -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256 -
pkeyopt rsa_mgf1_md:sha256 -in msg.txt -out from_openssl.bin
(ciphertext written)
```

5. A Simple PKI Environment

A **Public Key Infrastructure** answers the question encryption alone cannot: *whose* public key is this? A **Certificate Authority (CA)** checks an identity and signs a **certificate** binding that identity to a public key. Anyone who trusts the CA can then trust the key. HVMC's PKI has two tiers, like most enterprise PKIs:

```
HVMC Root CA R1            RSA-4096  SHA-384  10 years  pathLen 1   offline — signs only CA certificates and CRLs
  └── HVMC Issuing CA 1     RSA-3072  SHA-384   5 years  pathLen 0   online — issues end-entity certificates
        ├── portal.hvmc.example   RSA-2048  SHA-256  397 days  serverAuth   DNS: portal.hvmc.example, ehr.hvmc.example
        └── Sarah Chen            RSA-2048  SHA-256  365 days  clientAuth, emailProtection   sarah.chen@hvmc.example
```

Why two tiers? The root key is the anchor of all trust, so it stays **offline** and is used only a few times in its life. Day-to-day issuing happens with the issuing CA; if that key were compromised, HVMC would revoke it with the root and build a new one, without replacing the root in every device's trust store. **pathLen 0** on the issuing CA means it can sign only end-entity certificates, never another CA.

5.1 Certificates issued

Subject	Issuer	Key	Signature	Type	pathLen	Valid until
HVMC Root CA R1	HVMC Root CA R1	RSA-4096	sha384-RSA	CA	1	2036-10-23
HVMC Issuing CA 1	HVMC Root CA R1	RSA-3072	sha384-RSA	CA	0	2031-10-25
portal.hvmc.example	HVMC Issuing CA 1	RSA-2048	sha256-RSA	end entity	—	2027-11-27
Sarah Chen	HVMC Issuing CA 1	RSA-2048	sha256-RSA	end entity	—	2027-10-26

5.2 Issuance: CSR and policy checks

The requester generates its own key pair and sends a **Certificate Signing Request (CSR)**, which is signed with the private key to prove possession; the private key never leaves the requester. Before signing, `IssuingCA.check_policy()` acts as the **registration authority**: it verifies the CSR signature, requires RSA ≥ 2048 bits, allows DNS names only under `.hvmc.example` (no wildcards) and email addresses only under `@hvmc.example`. Certificates get a **159-bit random serial**, the matching key usage and extended key usage, subject/authority key identifiers and a CRL distribution point. Each issuance is recorded in `pki/ca_register.json` (requester, profile, dates, status) — the equivalent of OpenSSL's `index.txt`.

```
CSR for portal.evil.example  -> rejected: DNS name portal.evil.example is outside .hvmc.example (or a wildcard)
CSR for *.hvmc.example       -> rejected: DNS name *.hvmc.example is outside .hvmc.example (or a wildcard)
CSR for RSA-1024 key         -> rejected: RSA-1024 is below the HVMC minimum of 2048 bits
CSR for someone@gmail.com    -> rejected: email someone@gmail.com is outside @hvmc.example
```

```
    def check_policy(self, csr, profile):
        """Registration-authority checks before anything is signed."""
        if not csr.is_signature_valid:
            raise PolicyViolation("CSR signature does not verify (requester does not hold the key)")
        bits = csr.public_key().key_size
        if bits < 2048:
            raise PolicyViolation(f"RSA-{bits} is below the HVMC minimum of 2048 bits")
        try:
            sans = csr.extensions.get_extension_for_class(x509.SubjectAlternativeName).value
        except x509.ExtensionNotFound:
            raise PolicyViolation("CSR has no subjectAltName")
        dns, emails = sans.get_values_for_type(x509.DNSName), sans.get_values_for_type(x509.RFC822Name)
        for d in dns:
            if not d.endswith(ALLOWED_DNS_SUFFIX) or "*" in d:
                raise PolicyViolation(f"DNS name {d} is outside {ALLOWED_DNS_SUFFIX} (or a wildcard)")
        for e in emails:
            if not e.endswith(ALLOWED_EMAIL_DOMAIN):
                raise PolicyViolation(f"email {e} is outside {ALLOWED_EMAIL_DOMAIN}")
        if profile == "tls-server" and not dns:
            raise PolicyViolation("server certificate needs at least one DNS name")
        if profile == "client-auth" and not emails:
            raise PolicyViolation("client certificate needs an email address")
        return sans
```

5.3 Certificate contents (OpenSSL view)

```
$ openssl x509 -in certs/portal.hvmc.example.crt.pem -noout -text
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            57:29:f2:4e:8f:ea:64:1b:46:b1:5c:08:ce:88:a0:33:5d:e9:37:15
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=US, O=Horizon Valley Medical Center, OU=Information Security, CN=HVMC Issuing CA 1
        Validity
            Not Before: Oct 26 17:32:03 2026 GMT
            Not After : Nov 27 17:37:03 2027 GMT
        Subject: C=US, O=Horizon Valley Medical Center, OU=IT Infrastructure, CN=portal.hvmc.example
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                RSA Public-Key: (2048 bit)
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Subject Alternative Name:
                DNS:portal.hvmc.example, DNS:ehr.hvmc.example
            X509v3 Subject Key Identifier:
                3D:16:7E:49:B3:76:CC:01:C7:62:C1:14:2F:F5:2E:F8:71:01:66:0B
            X509v3 Authority Key Identifier:
                keyid:79:35:F1:D5:4C:AF:AC:6B:B1:95:63:2E:D7:83:6F:12:98:E4:E9:70

            X509v3 CRL Distribution Points:

                Full Name:
                  URI:http://pki.hvmc.example/crl/hvmc-issuing-ca-1.crl
```

5.4 Revocation lists

Each CA publishes a signed **CRL**: the issuing CA weekly (next update in 7 days), the offline root every 180 days. Both are empty today; revoking a certificate and checking the CRL during validation is covered in the Digital Signatures report, which builds on this PKI.

```
$ openssl crl -in crl/hvmc-issuing-ca-1.crl.pem -noout -text
Certificate Revocation List (CRL):
        Version 2 (0x1)
    Signature Algorithm: sha256WithRSAEncryption
        Issuer: /C=US/O=Horizon Valley Medical Center/OU=Information Security/CN=HVMC Issuing CA 1
        Last Update: Oct 26 17:37:03 2026 GMT
        Next Update: Nov  2 17:37:03 2026 GMT
        CRL extensions:
            X509v3 CRL Number:
                1
            X509v3 Authority Key Identifier:
                keyid:79:35:F1:D5:4C:AF:AC:6B:B1:95:63:2E:D7:83:6F:12:98:E4:E9:70

No Revoked Certificates.
    Signature Algorithm: sha256WithRSAEncryption
```

5.5 Certificate path validation

`validate_path()` implements a simplified version of RFC 5280 §6: it builds the chain from the leaf to a trusted root, then checks every certificate's validity dates, its signature (twice — with the library and with the from-scratch PKCS #1 v1.5 verifier, which rebuilds the expected encoding and compares it whole), CA flags and key usage for issuers, the path-length limit, the CRL, the extended key usage and the hostname.

Check	Detail	Result
path ends at a trusted root	CN=HVMC Root CA R1,OU=Information Security,O=Horizon Valley Medical Center,C=US	**PASS**
portal.hvmc.example: inside validity period	2026-10-26 to 2027-11-27	**PASS**
portal.hvmc.example: signature (library / from-scratch)	library=True, from-scratch=True	**PASS**
portal.hvmc.example: CRL signature		**PASS**
portal.hvmc.example: CRL is current	next update 2026-11-02 17:37:03+00:00	**PASS**

Check	Detail	Result
portal.hvmc.example: not revoked		**PASS**
HVMC Issuing CA 1: inside validity period	2026-10-26 to 2031-10-25	**PASS**
HVMC Issuing CA 1: signature (library / from-scratch)	library=True, from-scratch=True	**PASS**
HVMC Issuing CA 1: basicConstraints CA=TRUE, keyCertSign	pathLen=0	**PASS**
HVMC Issuing CA 1: path length constraint	0 CA(s) below, limit 0	**PASS**
HVMC Issuing CA 1: CRL signature		**PASS**
HVMC Issuing CA 1: CRL is current	next update 2027-04-24 17:37:03+00:00	**PASS**
HVMC Issuing CA 1: not revoked		**PASS**
HVMC Root CA R1: inside validity period	2026-10-26 to 2036-10-23	**PASS**
HVMC Root CA R1: signature (library / from-scratch)	library=True, from-scratch=True	**PASS**
HVMC Root CA R1: basicConstraints CA=TRUE, keyCertSign	pathLen=1	**PASS**
HVMC Root CA R1: path length constraint	1 CA(s) below, limit 1	**PASS**
leaf: extendedKeyUsage includes serverAuth		**PASS**
leaf: hostname portal.hvmc.example in subjectAltName	portal.hvmc.example, ehr.hvmc.example	**PASS**

Three independent verifiers agree: the from-scratch validator, the library's RFC 5280 verifier (`cryptography.x509.verification`) and `openssl verify`:

```
$ openssl verify -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -purpose sslserver
certs/portal.hvmc.example.crt.pem
certs/portal.hvmc.example.crt.pem: OK
$ openssl verify -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -purpose sslclient
certs/sarah.chen.crt.pem
certs/sarah.chen.crt.pem: OK
$ openssl verify -CAfile certs/hvmc-root-ca-r1.crt.pem -untrusted certs/hvmc-issuing-ca-1.crt.pem -purpose sslserver
certs/sarah.chen.crt.pem
C = US, O = Horizon Valley Medical Center, OU = Doctors, CN = Sarah Chen, emailAddress = sarah.chen@hvmc.example
error 26 at 0 depth lookup:unsupported certificate purpose
certs/sarah.chen.crt.pem: verification failed: 26 (unsupported certificate purpose)
```

5.6 Negative tests

Attempt	Result
Portal certificate used for `pharmacy.hvmc.example`	Rejected — hostname not in subjectAltName
Validation 400 days from now	Rejected — certificate expired
Sarah's client certificate presented as a server	Rejected — no serverAuth EKU (OpenSSL: *unsupported certificate purpose*)
Issuing CA certificate missing from the chain	Rejected — no path to a trusted root
Impostor root named "HVMC Root CA R1" but with an attacker's key	Rejected — signatures do not verify
Certificate "issued" by the portal's own key	Rejected — portal is not a CA (pathLen 0 on the issuing CA)
CSRs for `portal.evil.example`, `*.hvmc.example`, an RSA-1024 key, a Gmail address	Refused by the issuance policy before signing

6. Secure Key Exchange

Key exchange lets two parties derive the same secret over a network that an attacker can watch. The HVMC Cryptographic Fundamentals report implemented classic finite-field Diffie-Hellman (RFC 3526, 2048-bit). This section uses its modern elliptic-curve form, **X25519**, and — more importantly — shows how to make the exchange **authenticated** with the PKI from section 5.

6.1 X25519 and HKDF from scratch

X25519 (RFC 7748) works on Curve25519 over the prime field $2^{255} - 19$. A private key is 32 random bytes, **clamped** (low 3 bits cleared so the key is a multiple of the cofactor 8, bit 254 set so every key takes the same number of steps). The public key is the

private scalar times the base point $u = 9$, computed with the **Montgomery ladder**, which performs the same operations for every key bit (a conditional swap instead of a branch). Both sides compute $a \cdot (b \cdot G) = b \cdot (a \cdot G)$. An all-zero result means the peer sent a low-order point, and the exchange is aborted.

HKDF (RFC 5869) turns the shared secret into keys: *Extract* = HMAC(salt, secret) concentrates the entropy; *Expand* = HMAC(PRK, T(i−1) ‖ info ‖ i) produces as many independent keys as needed, separated by the *info* label.

```
def x25519(k, u):
    """Montgomery ladder on Curve25519, written as in RFC 7748 section 5.
    Each step does the same work whatever the key bit is, with a conditional swap."""
    k, x1 = _decode_scalar(k), _decode_u(u)
    x2, z2, x3, z3, swap = 1, 0, x1, 1, 0
    for t in reversed(range(255)):
        kt = (k >> t) & 1
        swap ^= kt
        if swap:
            x2, x3, z2, z3 = x3, x2, z3, z2
        swap = kt
        a, b = (x2 + z2) % P, (x2 - z2) % P
        aa, bb = a * a % P, b * b % P
        e = (aa - bb) % P
        c, d = (x3 + z3) % P, (x3 - z3) % P
        da, cb = d * a % P, c * b % P
        x3 = (da + cb) ** 2 % P
        z3 = x1 * (da - cb) ** 2 % P
        x2 = aa * bb % P
        z2 = e * (aa + A24 * e) % P
    if swap:
        x2, z2 = x3, z3
    return (x2 * pow(z2, P - 2, P) % P).to_bytes(32, "little")
```

Test vector	Source	Result
Scalar × u-coordinate (vector 1)	RFC 7748 §5.2	**PASS**
Alice and Bob public keys and shared secret	RFC 7748 §6.1	**PASS**
Public keys and shared secrets match the library's X25519 (3 random key pairs)	cryptography	**PASS**
Low-order point (u = 0) rejected	RFC 7748 §6.1	**PASS**
HKDF-SHA256 test case 1 (PRK and 42-byte OKM), and matches the library's HKDF	RFC 5869 A.1	**PASS**

6.2 The problem: an unauthenticated exchange

Diffie-Hellman on its own proves nothing about *who* sent a public key. An attacker in the middle (on hospital Wi-Fi, for example) can replace both public keys with its own and complete one exchange with each side. The demo shows the client and server end up with **different** keys, and the attacker holds **both** — it can decrypt, read and re-encrypt every message without either side noticing.

6.3 The HVMC design: certificate-authenticated ephemeral exchange

The handshake follows the structure of TLS 1.3 (simplified for teaching). Each side signs a hash of the whole conversation (the **transcript**) with its certificate key, so replacing any message breaks the signature. The client authenticates too (mutual TLS), using Sarah Chen's client certificate.

Client (Sarah)		Server (portal)
new X25519 key pair	→ ClientHello: ephemeral public key, nonce, server name	
	← ServerHello: ephemeral public key, nonce, certificate, RSA-PSS signature over the transcript	new X25519 key pair
validate certificate path to HVMC Root CA R1, hostname, serverAuth; verify signature		
	→ client certificate + RSA-PSS signature over the transcript	validate path, clientAuth; verify signature
X25519(own, peer) → HKDF	same 32-byte shared secret on both sides; two AES-256-GCM keys	X25519(own, peer) → HKDF
delete ephemeral private key	⇄ encrypted records (counter nonces)	delete ephemeral private key

The session keys come from `HKDF(salt = transcript hash, secret = X25519 output)` with one key per direction. Each record is sealed with AES-256-GCM using a counter as the nonce, so a record that is modified, replayed or reordered fails authentication.

```
=== Demo 5: Secure key exchange ===
  unauthenticated X25519 with an attacker in the middle: client and server keys differ = True; attacker holds the client's key = True and
the server's key = True -> it can read and re-encrypt everything
  1. ClientHello: X25519 ephemeral key + 32-byte nonce, SNI portal.hvmc.example
  2. ServerHello: X25519 ephemeral key + nonce + certificate + RSA-PSS signature over transcript
  3. Client: server certificate path valid (19 checks) and transcript signature verified
  4. Server: client certificate valid for clientAuth (Sarah Chen) and signature verified
  5. Both: HKDF-SHA256 over the X25519 secret and transcript hash -> two AES-256-GCM keys; ephemeral private keys deleted
  Sarah -> portal: 55-byte record 4d0f0a1191e7b23a3849b20b… -> server reads 'GET /ehr/patients/HVMC-004417/discharge'
  portal -> Sarah: 185-byte record -> client reads 169 bytes of the record
  replaying Sarah's first record -> rejected (sequence-number nonce, GCM tag fails)
  attacker swaps the server's ephemeral key -> server signature over the handshake does not verify (key substitution?)
  attacker presents its own certificate for portal.hvmc.example -> server certificate rejected: portal.hvmc.example: signature (library /
from-scratch)
  RSA key transport: a recorded session + the server's RSA key stolen later -> session key recovered = True (no forward secrecy)
  ephemeral X25519: the recording holds only public values and signatures; the ephemeral private keys were deleted, so the stolen RSA key
cannot rebuild the session key (forward secrecy)
```

6.4 Forward secrecy: RSA key transport vs ephemeral X25519

	RSA key transport (old TLS-RSA)	Ephemeral X25519 + certificate signatures (HVMC design)
How the session key is agreed	Client picks it and sends it RSA-OAEP-encrypted to the server's long-term key	Both sides contribute a fresh ephemeral key; the long-term RSA key only signs
Attacker records traffic, steals the server's RSA key a year later	**Session key recovered = True** — every recorded session can be decrypted	Nothing to recover: the recording holds only public values; the ephemeral private keys were deleted after the handshake
Forward secrecy	**No**	**Yes** — the reason TLS 1.3 removed RSA key transport

7. Security Considerations and Best Practices

Area	HVMC practice	Shown in
Key size	RSA ≥ 2048 bits (enforced at generation and in the CSR policy); 3072+ for keys that must last past 2030	2.2, 5.2
Randomness	All keys, seeds, nonces and serials from the OS CSPRNG (`secrets`)	2.2, 4.2
Padding	Never textbook RSA; OAEP-SHA-256 for encryption, PSS for new signatures	4.1, 4.2, 6.3
Bulk data	Hybrid encryption: RSA wraps a data key, AES-256-GCM protects the data, with associated data	4.3
Private key storage	Encrypted PKCS #8, mode 0600, never in Git; production keys in HSM / Key Vault, non-exportable	3.2
Lifecycle	Inventory with owner and cryptoperiod; rotation by rewrapping; retire then destroy	3.3, 3.4

Area	HVMC practice	Shown in
PKI	Offline root, pathLen limits, short end-entity lifetimes, RA policy checks before signing, CRLs	5
Validation	Full chain, dates, CA flags, EKU, hostname, revocation — never "accept any certificate"	5.5, 5.6
Key exchange	Ephemeral (EC)DHE for forward secrecy, authenticated by certificate signatures over the transcript	6
Errors and side channels	One generic OAEP error; RSA blinding; Montgomery ladder with conditional swaps	2.3, 4.2, 6.1

7.1 Limitations of this lab

- The from-scratch code is for learning. Python integers are not constant-time, so it is not side-channel safe; HVMC production systems use TLS 1.3 and vetted libraries.
- The root CA key is "offline" by convention only; it sits in the same encrypted folder as the other keys. A real deployment keeps it in an offline HSM in a key ceremony with two people present.
- The handshake is a teaching model of TLS 1.3; it has no version negotiation, session resumption or alerts. Real TLS on a web server, SSH keys and a Wireshark analysis are the subject of the Cryptographic Protocols rubric.
- Revocation is published but not exercised; certificate revocation, renewal and the trust model are documented in the Digital Signatures report, which reuses this PKI.

8. Conclusion

HVMC now has a working public key foundation. RSA keys are generated from first principles and proven valid by the library and OpenSSL; they are stored encrypted, inventoried and rotated under a written policy. RSA-OAEP and hybrid encryption protect patient records correctly where textbook RSA fails. A two-tier PKI issues server and clinician certificates under policy, and three independent verifiers agree on every chain. Finally, an X25519 exchange authenticated by those certificates gives HVMC confidential, tamper-evident channels with forward secrecy, and stops the man-in-the-middle attack that defeats unauthenticated Diffie-Hellman.

Appendix A. Test Results

Run with `python3 test_and_demo.py` (Python 3.9.6 on Darwin arm64, 2026-10-26 13:37; cryptography package: 47.0.0; LibreSSL 3.3.6).

Test	Group	Result
`test_crt_and_blinding_match_plain_decryption`	RSA	**PASS**
`test_keygen_2048_properties_and_library_accepts`	RSA	**PASS**
`test_number_theory`	RSA	**PASS**
`test_oaep_interop_with_library_both_ways`	RSA	**PASS**
`test_oaep_round_trip_random_and_limits`	RSA	**PASS**
`test_pkcs1v15_verify_on_real_certificates`	RSA	**PASS**
`test_textbook_example`	RSA	**PASS**
`test_encrypted_storage_permissions_and_passphrase`	KeyManagement	**PASS**
`test_expiry_enforced`	KeyManagement	**PASS**
`test_hybrid_rotation_and_lifecycle`	KeyManagement	**PASS**
`test_chain_valid_mine_and_library`	PKI	**PASS**
`test_csr_policy`	PKI	**PASS**
`test_rejections`	PKI	**PASS**
`test_authenticated_handshake`	KeyExchange	**PASS**
`test_handshake_rejects_attacks`	KeyExchange	**PASS**
`test_hkdf_rfc5869`	KeyExchange	**PASS**
`test_rfc7748_vectors`	KeyExchange	**PASS**
`test_unauthenticated_exchange_is_open_to_mitm`	KeyExchange	**PASS**
`test_x25519_matches_library`	KeyExchange	**PASS**

Appendix B. Files

Published in `cryptography_1/public_key_crypto/`. The private keys and passphrase in `pki/private/` are created by the script and excluded by `.gitignore`.

```
.gitignore
key_exchange.py
key_management.py
openssl_checks.txt
pki.py
results.json
results.txt
rsa.py
sample_envelope_v2.json
test_and_demo.py
pki/ca_register.json
pki/key_inventory.json
pki/certs/hvmc-chain.pem
pki/certs/hvmc-issuing-ca-1.crt.pem
pki/certs/hvmc-root-ca-r1.crt.pem
pki/certs/portal.hvmc.example.crt.pem
pki/certs/sarah.chen.crt.pem
pki/crl/hvmc-issuing-ca-1.crl.pem
pki/crl/hvmc-root-ca-r1.crl.pem
pki/csr/hvmc-issuing-ca-1.csr.pem
pki/csr/portal.hvmc.example.csr.pem
pki/csr/sarah.chen.csr.pem
pki/public/ehr-data-key-v1.pub.pem
pki/public/ehr-data-key-v2.pub.pem
pki/public/hvmc-issuing-ca-1.pub.pem
pki/public/hvmc-root-ca-r1.pub.pem
pki/public/portal-tls-2026.pub.pem
pki/public/sarah-chen-client-2026.pub.pem
```

Appendix C. References

- RFC 8017 — PKCS #1: RSA Cryptography Specifications v2.2 (OAEP, PSS, PKCS #1 v1.5)
- FIPS 186-5 — Digital Signature Standard (RSA key-pair generation criteria)
- NIST SP 800-57 Part 1 Rev. 5 — Recommendation for Key Management (security strengths, cryptoperiods)
- RFC 5280 — Internet X.509 PKI Certificate and CRL Profile (path validation)
- RFC 7748 — Elliptic Curves for Security (X25519); RFC 5869 — HKDF; RFC 8446 — TLS 1.3
- CA/Browser Forum Baseline Requirements (serial-number entropy, 397-day TLS certificates)
- HIPAA Security Rule 45 CFR §164.312(d) and (e)(1)